Types construits

COURS

Partie 1 : Introduction

En Python, on dispose de **types de base** comme :

— les entiers (**int**) ;

— les nombres flottants (**float**) ;

— les valeurs booléennes (**bool**) ;

— les chaînes de caractères (**str**).

Ces types permettent de représenter des valeurs simples : un nombre, un mot, une valeur logique, etc. Cependant, dans la plupart des problèmes, les informations ne se présentent pas isolées mais **regroupées** : liste d'élèves, coordonnées d'un point, données d'un capteur, notes d'un devoir, informations sur un utilisateur...

C'est pourquoi Python (comme tous les langages) permet de créer des **types construits** : des objets capables de contenir **plusieurs valeurs**, organisées de façon cohérente.

Intérêts des types construits :

— **Regrouper des données liées** : les coordonnées d'un point (x, y), les informations d'un élève, les mesures d'une expérience.

— **Faciliter les traitements** : parcourir une liste de notes, trier, chercher, compter...

— **Rendre les programmes plus lisibles** : une structure bien choisie rend le code plus simple et plus clair.

— **Représenter des données complexes** du monde réel : tables de données, matrices, enregistrements, etc.

Dans ce chapitre, nous étudions trois types construits essentiels :

— les **p-uplets** (ou *tuples*) → regrouper des valeurs non modifiables ;

— les **listes** → stocker des éléments modifiables et itérables ;

— les **dictionnaires** → associer des *clés* et des *valeurs*.

Ils permettent de manipuler efficacement des données structurées, indispensables dans toute application informatique. Chacun de ces types a ses utilités et spécificités, il faut donc choisir correctement le type utilisé selon le problème qui nous est posé.

Partie 2 : Les p-uplets (tuples)

Un **p-uplet** (ou **tuple** en Python) est un ensemble ordonné de valeurs, placées les unes à la suite des autres. Le terme de **p-uplet** vient du fait que selon le nombre de valeurs qu'il contient, son nom peut être plus précis : `doublet, triplet, etc`.

Un tuple peut contenir des valeurs :

— de n'importe quel type ;

— possiblement toutes différentes (entier, chaîne, booléen...).

2.1 Créer un tuple

Créer un **tuple** revient à lister l'ensemble des valeurs que l'on veut stocker entre **parenthèses**, séparées par des **virgules**.

```
t = (12, "NSI", True, 3.14)
```

Une spécificité du tuple est que c'est un type **non mutable**, cela veut dire qu'une fois créé, on ne peut pas le modifier. On peut cependant en créer un nouveau par dessus ou en créer plusieurs.

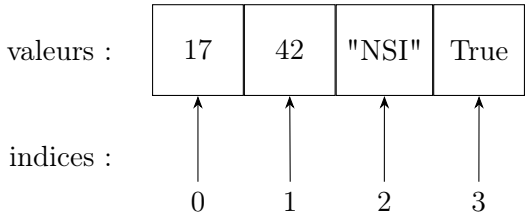
```
t = (1, 2, 3)
t2 = (3, 6, 7)   # creation d'un nouveau tuple
t2 = (2, 2, 4)   # creation d'un nouveau tuple en ecrasant l'ancien
print(t2) # (2, 2, 4)
```

2.2 Indices et valeurs

Les éléments d'un tuple sont identifiables grâce à leur `position` dans le tuple, cette `position` est appelée **indice**, chaque élément possède donc une **valeur** et un **indice**.

Voici un exemple de tuple et la représentation de ses valeurs et indices :

```
t = (17, 42, "NSI", True)
```



Remarque

Comme souvent en informatique, on commence à compter les indices à partir de **0**.

Attention

On distingue l'**indice** (la position) et la **valeur** (le contenu).

2.3 Accéder aux valeurs

C'est cette notion d'indice qui va nous permettre d'accéder aux valeurs d'un tuple. Par exemple, pour un tuple `t`, si je veux accéder à la valeur de l'indice `i` nous suffit d'utiliser cette syntaxe :

`t[i]`

Application

Créer un tuple `identite` contenant deux valeurs : votre prénom et votre nom. Récupérer le prénom contenu dans le tuple dans une variable `prenom`, faites pareil pour le nom dans une variable `nom` puis imprimer ces deux variables.

.....

.....

.....

.....

.....

2.4 Parcourir un tuple

Il est très fréquent de vouloir traiter successivement tous les éléments d'un tuple : afficher chaque valeur, effectuer un calcul, rechercher un élément, etc. Python propose deux manières principales de parcourir un tuple :

— le **parcours par valeurs** : on récupère directement chaque élément du tuple ;

— le **parcours par indices** : on parcourt les positions (indices) et on accède aux valeurs via `t[i]`.

Les deux méthodes sont correctes, mais la première est plus simple et plus lisible lorsque l'indice n'est pas nécessaire.

Parcours par valeurs

Dans ce type de parcours, la variable de boucle prend successivement la **valeur de chaque élément** du tuple.

```
1  t = ("a", "b", "c")
2  for element in t:
3    print(element)
```

Voici le déroulement de la boucle :

Iteration	Valeur de `element`	Affichage
1	"a"	a
2	"b"	b
3	"c"	c

À chaque tour, `element` contient directement la valeur stockée dans le tuple.

Parcours par indices

Dans ce parcours, on utilise les positions du tuple. La boucle va donc faire varier l'indice `i`, et on accède à la valeur via `t[i]`.

```
1  t = ("a", "b", "c")
2  for i in range(len(t)):
3    print(i, "->", t[i])
```

Déroulement détaillé :

Iteration	Valeur de `i`	Valeur de `t[i]`	Affichage
1	0	"a"	a
2	1	"b"	b
3	2	"c"	c

Remarque

Ici, `len(t)` récupère la longueur de `t`. Cela permet de savoir directement combien d'éléments sont dans le tuple, pour que la boucle fasse le bon nombre d'itération.

Ce parcours est utile lorsqu'on a besoin :

— d'utiliser l'indice (ex : position d'un élément) ;

— d'accéder simultanément à plusieurs éléments à partir de leurs indices ;

— d'effectuer certains calculs dépendant de la position.

Application

Nous allons utiliser le tuple `t=(2, 4, 7, 3, 0, 7, 3)`, parcourez ce tuple par element et affichez `"trouvé"` dès que vous parcourez un élément valant **0**. Parcourez ensuite ce même tuple pour afficher `l'indice` actuel dès que vous trouvez un élément valant **3**.

.....

.....

.....

.....

.....

.....

.....

2.5 Utilisations particulières

En plus d'accéder à des valeurs, plusieurs manipulations sont possibles sur des tuples, en-voici une liste non-exhaustive :

— Il est possible de `concaténer` deux tuples

```
t = (4, 2, 4) + ("NSI", 4)  # t vaudra (4, 2, 4, "NSI", 4)
```

— Il est possible de faire une `multi-affectation` à partir d'un tuple, pour récupérer ses valeurs

```
t = (4, 2, 4, 9)
a, b, c, d = t  # a = 4, b = 2, c = 4, d = 9
```

— Une fonction qui renvoie plusieurs valeurs le fait sous forme de tuple

```
return 45, 12 # cela retourne le tuple (45, 12)
```

Remarque

Un tuple peut également être créer grâce à des variables, pas seulement des valeurs directement.
Exemple :

```
variable1 = 12
variable2 = "NSI"
t = (variable1, variable2)
```

Partie 3 : Les listes (ou tableaux)

Les listes sont similaires aux tuples, à quelques différences près :

- C'est un type mutable : on peut modifier une liste après l'avoir créée :
- La syntaxe est différente : on utilise des crochets `[]`.

Création, accès et modification

```
notes = [12, 15, 9] # On cree une liste de notes
notes[1] = 18 # On modifie la deuxieme note de la liste, contenue a l'indice 1
moins_bonne_note = notes[2] # On recupere la moins bonne note
print(notes)  # On affiche toutes les notes : [12, 18, 9]
```

Comme pour un tuple, on peut parcourir une liste à l'aide d'une boucle.

Remarque

Une liste peut être vide, par exemple : `l = []` fait de `l` une liste vide.

3.1 Performances

Quel intérêt d'avoir deux types différents ?

Avoir à la fois des listes et des tuples en Python répond à un vrai besoin de conception : les listes, parce qu'elles sont **mutables**, sont idéales pour représenter des collections qui doivent **évoluer** dans le temps, mais cette mutabilité implique un **coût en performance**, en **mémoire**, et une incertitude sur l'**intégrité** des données (elles peuvent être modifiées par erreur). Les tuples, eux, offrent une alternative **légère**, plus **rapide** et **sûre**, parfaite pour représenter des données fixes.

3.2 Utilisations particulières

Comme pour les tuples, diverses opérations sont possibles sur les listes, comme la concaténation et la multi-affectation pour récupérer les valeurs. Mais la vraie plus-value d'une liste vient de sa mutabilité. Il existe donc un ensemble de fonctions (appelées méthodes) pour les manipuler :

Chacune de ces méthodes suivantes est à utiliser à l'aide de la syntaxe suivante :

```
liste.methode(paramètre1, paramètre2, ...)
```

Méthodes essentielles

- `append(x)` : ajoute un élément `x` en fin de liste ;
- `insert(i,x)` : insère l'élément `x` à l'indice `i` ;

— `pop(i)` : retire l'élément à l'indice `i` et le renvoie. Par exemple : `var = liste.pop(8)` permet de récupérer le 9eme élément de la liste dans `var` ;

— `remove(x)` : retire le premier élément qui a pour valeur `x` de la liste.

Application

— Créer une liste avec la chaîne de caractères "NSI" ;

— Ajouter deux chaînes de caractères correspondant à tes deux autres spécialités ;

— Récupérer la valeur de l'élément à l'indice 2 dans la variable `spe` ;

— Supprimer l'élément "NSI" de ta liste.

.....

.....

.....

.....

.....

3.3 Créer une liste par compréhension

Python permet de créer des listes de manière concise grâce aux **listes en compréhension** (*list comprehensions*). C'est une façon compacte d'écrire une boucle qui construit une liste.

La forme générale est :

[expression for variable in sequence]

Cette expression signifie : *« Pour chaque valeur de la séquence, calculer l'expression et l'ajouter dans une nouvelle liste. »*

Exemple simple : carrés des nombres de 0 à 5

```
1  l = [x*x for x in range(6)]
```

Déroulement de la compréhension :

Iteration	Valeur de x	Expression x*x
1	0	0
2	1	1
3	2	4
4	3	9
5	4	16
6	5	25

Ainsi, la liste obtenue est :

`[0, 1, 4, 9, 16, 25]`

Remarque

Les listes en compréhension sont particulièrement pratiques pour construire rapidement des listes numériques mais cela reste une méthode plus compliquée à utiliser qu'a comprendre.

3.4 Listes de listes

Comme une liste peut contenir des éléments de n'importe quel type, elle peut aussi contenir d'autres listes. On parle alors de **liste à deux dimensions** (comme un tableau à deux entrées). C'est un moyen simple de représenter un tableau, une grille, une matrice (tableau de nombres en mathématiques), etc.

Exemple de création d'une liste de listes

```
1  mat = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

Visuellement :

1	2	3
4	5	6
7	8	9

Chaque ligne est une liste, et l'ensemble de ces lignes forme une autre liste.

Accéder à une valeur

L'accès se fait à l'aide de deux indices :

— le premier indice : choisit la ligne ;

— le second indice : choisit la colonne dans cette ligne.

```
1  print(mat[0][1])  # 2
2  print(mat[2][0])  # 7
```

Déroulement de l'accès `mat[0][1]` :

Expression	Résultat
mat[0]	[1, 2, 3]
mat[0][1]	2

Application

Afficher la somme des valeurs des coins du tableau ci-dessus à l'aide de la liste de listes python qui le représente.

. . . . . .

. . . . . .

Remarque

Rien n'empêche de créer des listes à plus de deux dimensions : `mat[ligne][colonne][profondeur]` et ainsi de suite.
On peut donc imaginer des structures à 3D, 4D, etc.

Parcourir une liste à deux dimensions

Pour traiter toutes les valeurs d'une liste de listes, on utilise généralement deux boucles imbriquées :

— la première parcourt les **lignes** ;

— la seconde parcourt les **colonnes** d'une ligne.

```
1  mat = [[1, 2, 3], [4, 5, 6],[7, 8, 9]]
2
3  for ligne in mat:
4    for valeur in ligne:
5      print(valeur)
```

Dans ce parcours :

— la variable `ligne` prend successivement chaque liste interne ;

— la variable `valeur` prend chacun des éléments de cette ligne.

Déroulement du parcours

Iteration	Valeur de `ligne`	Valeur de `valeur`
1	[1, 2, 3]	1
2	[1, 2, 3]	2
3	[1, 2, 3]	3
4	[4, 5, 6]	4
5	[4, 5, 6]	5
6	[4, 5, 6]	6
7	[7, 8, 9]	7
8	[7, 8, 9]	8
9	[7, 8, 9]	9

Parcours par indices

On peut aussi parcourir une liste de listes à l'aide des indices :

```
1  for i in range(len(mat)): # indice de ligne
2    for j in range(len(mat[i])):  # indice de colonne
3      print(mat[i][j])
```

Déroulement simplifié :

Iteration	i	j
1	0	0
2	0	1
3	0	2
4	1	0
5	1	1
6	1	2
7	2	0
8	2	1
9	2	2

Dans ce cas, les valeurs sont récupérées via `mat[i][j]`. Ce type de parcours est utile lorsque l'on a besoin explicitement des indices.

Application

On considère le tableau suivant représentant des températures mesurées sur trois jours (et quatre moments de la journée) :

	Matin	Midi	16h	Soir
Jour 1	12	18	20	15
Jour 2	10	17	22	14
Jour 3	11	16	21	13

1. Reproduire ce tableau sous forme d'une liste de listes Python appelée `temp`.
2. Afficher toutes les valeurs au moyen d'un **parcours par valeurs**
3. Afficher toutes les valeurs en affichant également leur position, au moyen d'un **parcours par indices**
4. Trouver et afficher la **température la plus élevée** du tableau.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

3.5 Autres méthodes pour les listes

Il existe un certain nombre de méthodes et de fonctions permettant d'effectuer des traitements courants sur les listes. Ces outils évitent d'écrire des boucles complexes et rendent le code plus lisible.

Trier une liste : `sort`

La méthode `sort()` permet de trier une liste (par ordre croissant par défaut).

```
L = [5, 2, 9, 1]
L.sort()
print(L)   # [1, 2, 5, 9]
```

Remarque

La liste est directement modifiée.

Minimum et maximum : `min` et `max`

Les fonctions `min()` et`max()` permettent de récupérer la plus petite et la plus grande valeur d'une liste.

```
L = [5, 2, 9, 1]
print(min(L))  # 1
print(max(L))  # 9
```

Inverser une liste : `reverse`

La méthode `reverse()` inverse l'ordre des éléments de la liste, là aussi **sur place**.

```
L = [1, 2, 3, 4]
L.reverse()
print(L)   # [4, 3, 2, 1]
```

Extraire une partie d'une liste : le slicing

Le **slicing** permet d'extraire une sous-liste à partir d'une liste existante, sans la modifier. La syntaxe générale est :

`liste[début:fin]`

- `début` est inclus ;
- `fin` est exclu.

```
L = [10, 20, 30, 40, 50]
print(L[1:4])   # [20, 30, 40]
```

Exemples courants :

```
print(L[:3])    # [10, 20, 30]  (du debut a l indice 2)
print(L[2:])    # [30, 40, 50]  (de l indice 2 a la fin)
```

Remarque

Le slicing cree toujours une nouvelle liste, sans modifier celle utilisée.

Copier une liste : `copy`

La méthode `copy()` permet de créer une **nouvelle liste** contenant les mêmes valeurs.

```
L1 = [1, 2, 3]
L2 = L1.copy()

print(L1)  # [1, 2, 3]
print(L2)  # [1, 2, 3]
```

Quel est l'intérêt du copy ?

Sans utiliser la méthode `copy()`, deux variables contenant deux listes peuvent faire référence à la **même liste** :

```
L1 = [1, 2, 3]
L2 = L1
L2.append(4)

print(L1)  # [1, 2, 3, 4]
```

Dans ce cas, modifier `L2` modifie aussi `L1`. La méthode `copy()` permet d'éviter ce problème en travaillant sur une vraie copie indépendante :

```
L1 = [1, 2, 3]
L2 = L1.copy()
L2.append(4)

print(L1)  # [1, 2, 3]
print(L2)  # [1, 2, 3, 4]
```

Application

On dispose de la liste suivante représentant des scores obtenus par des joueurs :
`scores = [18, 7, 12, 15, 9, 20, 14]`

1. Creer une **copie** de la liste `scores` appelee `scores_copie`.
2. Trier la liste `scores_copie` par ordre croissant.
3. Afficher la plus petite et la plus grande valeur de `scores_copie`.
4. Inverser l ordre des elements de `scores_copie`.
5. Extraire dans une nouvelle liste :
   — les trois premieres valeurs ;
   — les trois dernieres valeurs.

.....

.....

.....

.....

.....

.....

.....

Partie 4 : Les dictionnaires

Un **dictionnaire** associe une **clé** à une **valeur**. On utilise des accolades `{}`.

```
eleve = {
  "nom": "Durand",
  "age": 16,
  "note": 15
}

```

Accès et modification

```
print(eleve["nom"])     # Durand
eleve["note"] = 17      # modification
eleve["ville"] = "Lyon" # ajout

```

Méthodes clés

- `keys()` : renvoie les clés
- `values()` : renvoie les valeurs
- `items()` : renvoie couples (clé,valeur)

```
for cle,val in eleve.items():
print(cle, "->", val)
```

Schéma

Clé	nom	age	note
Valeur	Durand	16	17

Remarque

Les clés doivent être immuables (str, int, tuple...).

Application

Créer un dictionnaire représentant une image EXIF minimal : largeur, hauteur, format. Afficher chaque clé suivie de sa valeur.

.....

.....

.....

.....

.....

.....

Partie 5 : Synthèse

Type	Mutable ?	Notation	Exemple
Tuple	Non	()	(1,2,"a")
Liste	Oui	[]	[1,2,3]
Dictionnaire	Oui		"nom":"Ana"